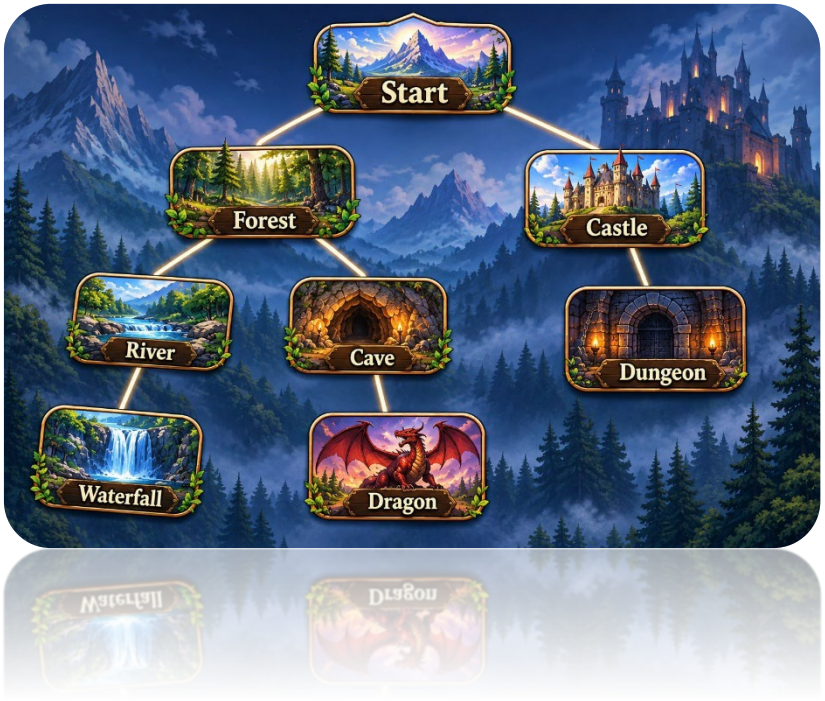
پاسخ سوال یک:

الف) پدر مرحله‌ی Dragon :

ساختار مراحل بازی به صورت زیر است:



برای رسیدن به مرحله‌ی Dragon ابتدا از مرحله‌ی Cave عبور می‌کنیم.

بنابراین پدر Dragon، مرحله‌ی Cave است.

ب) عمق Dragon :

مسیر Dragon تا ریشه به صورت زیر است:

Start → Forest → Cave → Dragon

در این مسیر ۳ یال وجود دارد، بنابراین:

Depth(Dragon) = 3

ج) ارتفاع درخت :

ارتفاع درخت برابر با بیشترین عمق موجود در آن است.

عمیق‌ترین مرحله، Dragon یا Waterfall است که عمق آن‌ها برابر ۳ است.

بنابراین:

Height = 3

د) پیمایشPreorder :

در پیمایش Preorderابتدا گره فعلی بررسی می‌شود و سپس فرزندان آن از چپ به راست پیمایش می‌شوند.

بنابراین ترتیب پیمایش به صورت زیر است:

Start → Forest → River → Waterfall → Cave → Dragon → Castle → Dungeon

پاسخ سوال دو:

فایل Q2.py را اجرا کنید

توضیحات:

نکته‌ی اصلی این مسئله این است که در پیمایش Preorder، اولین عنصر همیشه ریشه‌ی درخت است.

در مثال، اولین عدد 1 است؛ بنابراین 1 ریشه‌ی درخت است.

حالا جای 1 را در پیمایش Inorder پیدا می‌کنیم:

4 2 5 | 1 | 6 3 7

تمام عناصر سمت چپ 1 متعلق به زیردرخت چپ و تمام عناصر سمت راست آن متعلق به زیردرخت راست هستند.

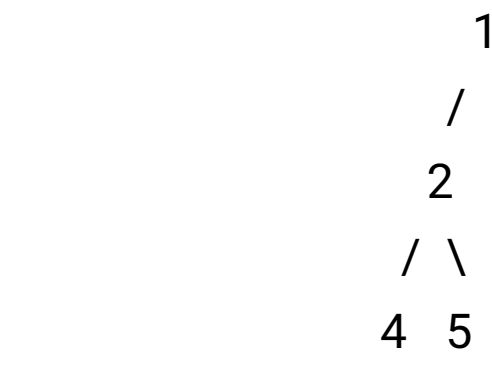
پس:

زیردرخت چپ: 4 2 5
زیردرخت راست: 6 3 7

اکنون همین تقسیم را در Preorder نیز انجام می‌دهیم. بعد از ریشه، سه عنصر مربوط به زیردرخت چپ قرار دارند:

2 4 5 | 3 6 7

بنابراین ساختار درخت به شکل زیر خواهد بود:



اکنون همین فرآیند را برای زیردرخت‌های چپ و راست ادامه می‌دهیم.

برای ساخت Postorder، ابتدا زیردرخت چپ، سپس زیردرخت راست و در پایان خود ریشه را قرار می‌دهیم.

در نتیجه:

4 5 2 6 7 3 1

مراحل حل

۱. اولین عنصر Preorder را به‌عنوان ریشه در نظر می‌گیریم.
۲. جای ریشه را در Inorder پیدا می‌کنیم.
۳. عناصر سمت چپ ریشه در Inorder، زیردرخت چپ را تشکیل می‌دهند.
۴. عناصر سمت راست ریشه، زیردرخت راست را تشکیل می‌دهند.
۵. با توجه به تعداد عناصر زیردرخت چپ، بخش مربوط به آن را از Preorder جدا می‌کنیم.
۶. همین فرآیند را به‌صورت بازگشتی برای دو زیردرخت ادامه می‌دهیم.

۷. هنگام برگشت از بازگشت، ابتدا زیردرخت چپ، سپس زیردرخت راست و در نهایت ریشه را قرار می‌دهیم تا Postorder ساخته شود.

پیچیدگی زمانی

اگر برای پیدا کردن محل هر ریشه، Inorder را در هر مرحله جست‌وجو کنیم، در بدترین حالت زمان اجرا می‌تواند $O(n^2)$ شود.

برای جلوگیری از این مسئله، محل هر مقدار در Inorder را از ابتدا در یک دیکشنری ذخیره می‌کنیم. در این صورت پیدا کردن محل هر ریشه در $O(1)$ انجام می‌شود.

بنابراین هر گره فقط یک بار پردازش شده و پیچیدگی زمانی کلی برابر است با:

$O(n)$

نکته: در کد پاسخ Q2.py برای جلوگیری از خطای عمق بازگشت، همین ایده با یک پشته و به‌صورت تکراری پیاده‌سازی شده است؛ این روش نیز تنها یک بار روی Preorder حرکت می‌کند و پیچیدگی آن $O(n)$ است.

پاسخ سوال سه:

فایل Q3.py را اجرا کنید

توضیحات:

برای حل مسئله یک BST می‌سازیم که هر گره یک مقدار و دو اشاره‌گر به فرزند چپ و راست خود دارد.

ویژگی اصلی BST این است که:

مقادیر زیردرخت چپ > مقدار گره > مقادیر زیردرخت راست

بنابراین هنگام جستجو یا اضافه‌کردن، با مقایسه‌ی مقدار موردنظر با گره فعلی می‌توانیم مشخص کنیم که باید به سمت چپ برویم یا راست.

۱. عملیات ADD

برای اضافه‌کردن X از ریشه شروع می‌کنیم.

اگر:

X < node.value

باشد، به زیردرخت چپ می‌رویم و اگر:

X > node.value

باشد، به زیردرخت راست می‌رویم.

اگر به محل خالی برسیم، گره جدید را همان‌جا قرار می‌دهیم.

اگر مقدار X را در درخت پیدا کنیم، چون مقادیر تکراری مجاز نیستند، هیچ تغییری ایجاد نمی‌کنیم.

۲. عملیات FIND

جستجو نیز با همین ویژگی BST انجام می‌شود.

برای مثال، برای پیدا کردن 40 در درخت نمونه ابتدا به 50 می‌رسیم:

40 < 50

پس به سمت چپ می‌رویم و به 30 می‌رسیم:

40 > 30

بنابراین به سمت راست می‌رویم و به 40 می‌رسیم.

در نتیجه مقدار موردنظر پیدا شده است.

۳. عملیاتPRINT

برای چاپ مقادیر به ترتیب صعودی از پیمایش Inorder استفاده می‌کنیم:

Left → Root → Right

یکی از ویژگی‌های BST این است که پیمایش Inorder، مقادیر موجود در درخت را به ترتیب صعودی تولید می‌کند.

۴. عملیاتREMOVE

ابتدا با استفاده از ویژگی BST گره موردنظر را پیدا می‌کنیم. سپس سه حالت ممکن داریم.

حالت اول: گره بدون فرزند

اگر گره برگ باشد، کافی است آن را حذف کنیم و ارتباط آن با پدرش از بین برود.

حالت دوم: گره دارای یک فرزند

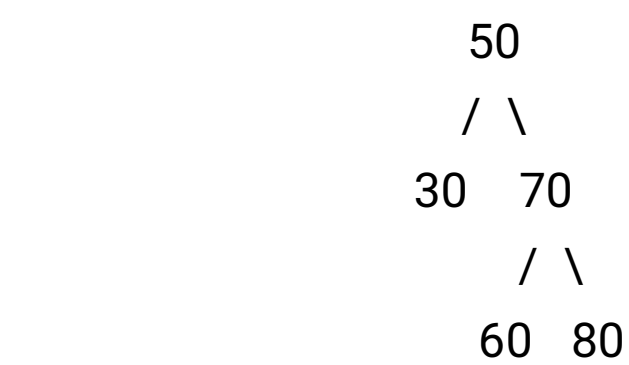
اگر گره فقط یک فرزند داشته باشد، همان فرزند جای گره حذف‌شده قرار می‌گیرد.

برای مثال اگر ۴۰ فقط یک فرزند (مثلاً ۳۵) داشته باشد، با حذف ۴۰، گره ۳۵ جای آن قرار می‌گیرد و مستقیماً به پدر ۴۰ وصل می‌شود.

حالت سوم: گره دارای دو فرزند

در این حالت می‌توانیم کوچک‌ترین مقدار در زیردرخت راست را پیدا کنیم و آن را به‌عنوان جانشین گره در نظر بگیریم.

برای مثال در درخت زیر:

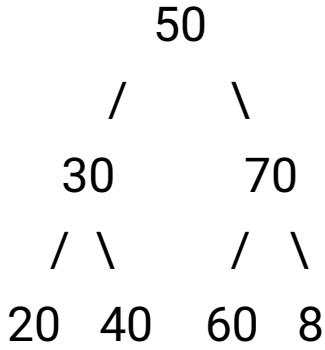


اگر 50 را حذف کنیم، کوچک‌ترین مقدار در زیردرخت راست آن 60 است.

بنابراین مقدار 50 را با 60 جایگزین کرده و سپس گره 60 را از محل قبلی حذف می‌کنیم.

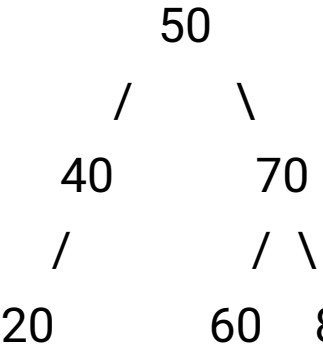
بررسی مثال

پس از عملیات‌های ADD، درخت به شکل زیر است:



با حذف30 ، این گره دو فرزند دارد. کوچک‌ترین مقدار در زیردرخت راست آن 40 است؛ بنابراین 40 جایگزین

30 می‌شود:



پس پیمایش Inorder برابر است با:

20 40 50 60 70 80

پیچیدگی زمانی

اگر ارتفاع درخت را h در نظر بگیریم:

- ADD برابر O(h)
- FIND برابر O(h)
- REMOVE برابر O(h)
- PRINT برابر O(n)

است.

در یک BST متعادل، ارتفاع تقریباً O(log n) است؛ اما اگر درخت به شکل زنجیره‌ای دربیاید، ارتفاع می‌تواند به O(n) برسد.

پاسخ سوال چهار:

فایل Q4.py را اجرا کنید

توضیحات:

برای حل مسئله ابتدا کد ایستگاه‌ها را به همان ترتیب ورودی داخل یک BST قرار می‌دهیم.

ویژگی اصلی BST این است که:

- تمام مقادیر زیردرخت چپ یک گره، کوچک‌تر از آن هستند .
- تمام مقادیر زیردرخت راست یک گره، بزرگ‌تر از آن هستند .

همین ویژگی باعث می‌شود برای پیدا کردن LCA لازم نباشد کل درخت را پیمایش کنیم.

۱. ساختBST

اعداد ورودی را یکی‌یکی در درخت قرار می‌دهیم.

برای هر مقدار:

- اگر از مقدار گره فعلی کوچک‌تر باشد، به سمت چپ می‌رویم .
- اگر بزرگ‌تر باشد، به سمت راست می‌رویم .

برای مثال با ورود اعداد:

50 30 80 20 40

ابتدا 50 ریشه می‌شود.

30 از 50 کوچک‌تر است، پس در سمت چپ قرار می‌گیرد.

80 از 50 بزرگ‌تر است، پس در سمت راست قرار می‌گیرد.

به همین شکل سایر مقادیر نیز در محل مناسب قرار می‌گیرند.

۲. بررسی وجود ایستگاه‌ها

قبل از پیدا کردن LCA باید مطمئن شویم هر دو ایستگاه در درخت وجود دارند.

برای جستجوی یک مقدار، از ریشه شروع می‌کنیم:

- اگر مقدار موردنظر با گره فعلی برابر باشد، آن را پیدا کرده‌ایم .
- اگر کوچک‌تر باشد، فقط به زیردرخت چپ می‌رویم .
- اگر بزرگ‌تر باشد، فقط به زیردرخت راست می‌رویم .

اگر به None برسیم، مقدار موردنظر در درخت وجود ندارد.

اگر یکی از a یا b وجود نداشته باشد، برای آن درخواست:

NOT FOUND

چاپ می‌کنیم.

۳. پیدا کردن LCA

حالا فرض کنید هر دو ایستگاه در درخت وجود دارند و در گره فعلی قرار داریم.

سه حالت داریم.

حالت اول: هر دو مقدار کوچک‌تر از گره فعلی باشند.

در این حالت هر دو در زیردرخت چپ قرار دارند، بنابراین LCA نیز در همان سمت است و به چپ می‌رویم.

حالت دوم: هر دو مقدار بزرگ‌تر از گره فعلی باشند.

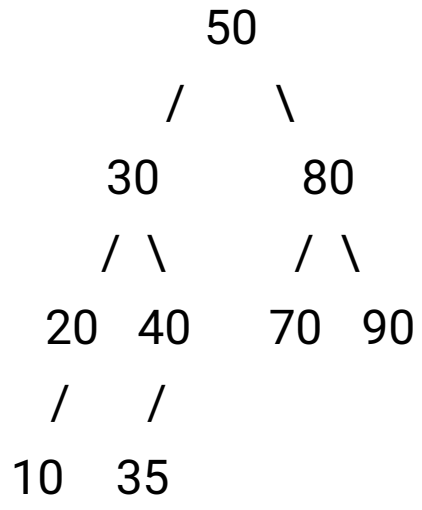
در این حالت هر دو در زیردرخت راست قرار دارند، بنابراین به سمت راست می‌رویم.

حالت سوم: یکی در سمت چپ و دیگری در سمت راست باشد.

در این حالت گره فعلی اولین گره‌ای است که مسیر دو ایستگاه در آن از یکدیگر جدا می‌شود؛ بنابراین خود گره فعلی LCA است.

همچنین اگر مقدار یکی از ایستگاه‌ها با گره فعلی برابر باشد، همان گره LCA خواهد بود.

برای مثال، درخت زیر را در نظر بگیرید:



برای 35 و 40:

هر دو در زیردرخت 40 قرار دارند و در نهایت 40 به‌عنوان LCA به دست می‌آید.

پس:

LCA(35, 40) = 40

برای 35 و 70:

یکی در سمت چپ 50 و دیگری در سمت راست 50 قرار دارد، بنابراین:

LCA(35, 70) = 50

۴. مراحل الگوریتم

برای هر درخواست:

۱. وجود aرا در BST بررسی می‌کنیم.
۲. وجود bرا در BST بررسی می‌کنیم.
۳. اگر یکی از آن‌ها وجود نداشت، NOT FOUNDچاپ می‌کنیم
۴. در غیر این صورت از ریشه شروع می‌کنیم.
۵. اگر هر دو مقدار از گره فعلی کوچک‌تر بودند، به چپ می‌رویم.
۶. اگر هر دو بزرگ‌تر بودند، به راست می‌رویم
۷. در غیر این صورت مقدار گره فعلی را به‌عنوان LCA چاپ می‌کنیم.

پیچیدگی زمانی

اگر ارتفاع درخت را hدر نظر بگیریم:

- ساخت BST: $O(n \times h)$
- جستجوی یک مقدار: $O(h)$
- پیدا کردن LCA: $O(h)$

بنابراین برای هر درخواست، زمان موردنیاز$O(h)$ است.

در یک BST متعادل، h تقریباً برابر$O(\log n)$ است؛ اما اگر درخت به شکل زنجیره‌ای ساخته شود، ارتفاع آن می‌تواند $O(n)$ باشد.

پاسخ سوال پنج:

فایل Q5.py را اجرا کنید

توضیحات:

در این مسئله به یک Augmented BSTنیاز داریم؛ یعنی علاوه بر ساختار معمولBST ، اطلاعات آماری موردنیاز را نیز در هر گره ذخیره کنیم.

هر گره شامل این اطلاعات است:

- price: قیمت سفارش
- count: تعداد سفارش‌هایی که دقیقاً همین قیمت را دارند
- size: تعداد کل سفارش‌ها در این گره و زیردرخت‌های آن
- sum: مجموع قیمت تمام سفارش‌ها در این گره و زیردرخت‌های آن

درBST ، قیمت‌های کوچک‌تر در سمت چپ و قیمت‌های بزرگ‌تر در سمت راست قرار می‌گیرند.

به‌روزرسانی size و sum:

برای هر گره داریم:

size = count + left.size + right.size

و:

sum = price × count + left.sum + right.sum

اگر فرزندی وجود نداشته باشد، مقدار size یا sum آن زیردرخت را صفر در نظر می‌گیریم.

بنابراین بعد از هر تغییر در یک گره، باید اطلاعات آن و سپس والدهایش به‌روزرسانی شوند.

۱. ثبت سفارش

در عملیات price 1، مانند یک درج معمولی BST حرکت می‌کنیم.

اگر:

price < node.price

باشد، به سمت چپ می‌رویم و اگر:

price > node.price

باشد، به سمت راست می‌رویم.

اگر قیمت برابر باشد، گره جدیدی ایجاد نمی‌کنیم و فقط:

count += 1

انجام می‌دهیم.

پس از تغییر، size و sum گره‌های مسیر باید به‌روزرسانی شوند.

۲. حذف سفارش

در عملیات price 2 ابتدا گره مربوط به قیمت را پیدا می‌کنیم.

اگر count > 1 باشد، فقط یک سفارش حذف شده است و کافی است:

```
count -= 1
```

انجام شود.

اگر count = 1 باشد، خود گره باید از BST حذف شود.

در این حالت سه وضعیت داریم:

- گره فرزند ندارد و مستقیماً حذف می‌شود.
- گره یک فرزند دارد و همان فرزند جایگزین آن می‌شود.
- گره دو فرزند دارد و کوچک‌ترین گره در زیردرخت راست به‌عنوان جانشین انتخاب می‌شود.

بعد از حذف نیز size و sum گره‌های مسیر باید به‌روزرسانی شوند.

۳. مجموع یک بازه

در عملیات ۳ L R، مجموع قیمت سفارش‌های بازه‌ی [L, R] برابر است با: (مجموع قیمت سفارش‌های کوچک‌تر یا مساوی R) منهای (مجموع قیمت سفارش‌های کوچک‌تر یا مساوی L-1).

برای محاسبه‌ی مجموع قیمت سفارش‌های کوچک‌تر یا مساوی x، از ریشه شروع می‌کنیم. اگر:

```
node.price <= x
```

باشد، قیمت این گره و تمام زیردرخت چپ آن در جمع وارد می‌شوند (با استفاده از price × count + left.sum) و برای ادامه به سمت راست می‌رویم.

اگر:

node.price > x

باشد، این گره و زیردرخت راست آن در جمع نمی‌آیند و فقط به سمت چپ می‌رویم.

به این ترتیب هر محاسبه فقط یک مسیر از ریشه تا یک برگ را طی می‌کند و زمان آن O(h) است.

در نتیجه: جواب = جمع(R) - جمع(L-1)

۴. پیدا کردن kاُمین سفارش گران‌تر

برای پیدا کردن kاُمین سفارش از سمت بزرگ‌ترین قیمت، ابتدا به زیردرخت راست نگاه می‌کنیم.

مقدار:

right.size

تعداد سفارش‌هایی را نشان می‌دهد که از گره فعلی گران‌تر هستند.

اگر:

right.size >= k

باشد، پاسخ در زیردرخت راست قرار دارد.

اگر kبزرگ‌تر از تعداد سفارش‌های سمت راست باشد اما در محدوده‌ی سفارش‌های همین گره قرار بگیرد، خود price پاسخ است.

در غیر این صورت باید به سمت چپ برویم و تعداد سفارش‌های بررسی‌نشده را به شکل زیر محاسبه کنیم:

k - right.size - count

۵. میانگین k سفارش گران‌تر

در این عملیات علاوه بر پیدا کردن k سفارش گران‌تر، مجموع قیمت آن‌ها نیز لازم است.

اگر تمام k سفارش موردنظر در زیردرخت راست باشند، مجموع آن‌ها مستقیماً از:

right.sum

به دست می‌آید.

اگر بخشی از سفارش‌ها متعلق به گره فعلی باشند، قیمت گره به تعداد موردنیاز در مجموع قرار می‌گیرد.

اگر هنوز تعدادی سفارش باقی مانده باشد، به زیردرخت چپ می‌رویم.

در نهایت میانگین با تقسیم صحیح محاسبه می‌شود:

average = total_sum // k

مراحل کلی حل

۱. یک BST دست‌ساز می‌سازیم .
۲. برای هر قیمت فقط یک گره ایجاد می‌کنیم و تعداد تکرارها را در countنگه می‌داریم.
۳. بعد از هر تغییر، size و sum را به‌روزرسانی می‌کنیم.
۴. برای مجموع بازه از ویژگی BST و فیلد sum استفاده می‌کنیم.
۵. برای kاُمین سفارش گران‌تر از size زیردرخت راست استفاده می‌کنیم.
۶. برای میانگین k سفارش گران‌تر، علاوه بر size از sum نیز استفاده می‌کنیم.

پیچیدگی زمانی

اگر ارتفاع درخت را h در نظر بگیریم:

عملیات	پیچیدگی
ثبت سفارش	O(h)
حذف سفارش	O(h)
مجموع بازه	O(h)
k اُمین سفارش گران‌تر	O(h)
میانگینk سفارش گران‌تر	O(h)

در حالت متعادل، ارتفاع درخت تقریباً O(log n) است؛ اما در بدترین حالت ممکن است ارتفاع به O(n) برسد.